

Fuel Ops Command Center

Fuel Supply Intelligence & Resilience Platform · BUP CSE Fest 2026 Hackathon

Observe → Detect → Predict → Decide → Simulate → Act → Monitor → Recover
All data comes from the BUP Fuel Supply Simulator (simulated, not real infrastructure)

THE PROBLEM

A simulated Bangladeshi fuel network: **2 depots, 4 stations, 6 routes, 3 fuels**, 15-minute ticks. Customers consume fuel every tick; supply ships only reach depots. Someone must decide **which depot sends how much of which fuel, on which route, and when** — through demand spikes, blocked routes, outages, late or reduced supply, and failing software.

Measured on the real simulator with **no action**: the first station runs dry at tick 65 (≈16 h) and only **46.1%** of demand is served over 2 days.

OUR APPROACH

- **Probe first, then design.** We measured the simulator before coding: demand model, lifecycle timing, dispatch rules, error formats.
- **One brain, one loop.** A single server-side control loop observes, plans and acts every tick; the UI only reads cached state.
- **Explainable optimisation, not a black box.** Hour-aware forecast → risk → constraint-safe planner → what-if → action.
- **Human in the loop where it matters.** Routine shipments are automatic; risky ones wait for the operator.
- **Resilience as a feature.** Every failure mode has a visible, tested behaviour.

KEY INSIGHT

We verified on **2,304 observations** that demand follows an exact pattern:

$$\text{demand/tick} = \text{daily profile} \div 96 \times \text{hour factor} \times \text{region factor} \times \text{spike multiplier} \times (\pm 10\% \text{ noise})$$

Industrial demand jumps **3.4×** at 06:00. A moving average lags that jump; our model sees it coming. We still calibrate online against real observations and report the forecast error.

WHAT MAKES IT UNIQUE

- **Event look-ahead:** reads *scheduled* crises and avoids a route before it is disrupted; pre-scales forecasts for upcoming demand spikes.
- **Shared-budget planner:** depot stock and per-tick dispatch capacity are deducted across all shipments in the same tick — no 409 rejections.
- **Auto-cancel doomed shipments:** pending shipments on a route about to be disrupted are cancelled and re-routed (fuel returns to the depot).
- **Stable idempotency keys:** retries and duplicate app instances can never double-ship.
- **Measured proof:** a built-in deterministic experiment runner compares strategies on identical scenarios.
- **Grounded AI:** OpenAI explains decisions and incidents from live facts only, never executes, and falls back to built-in text.

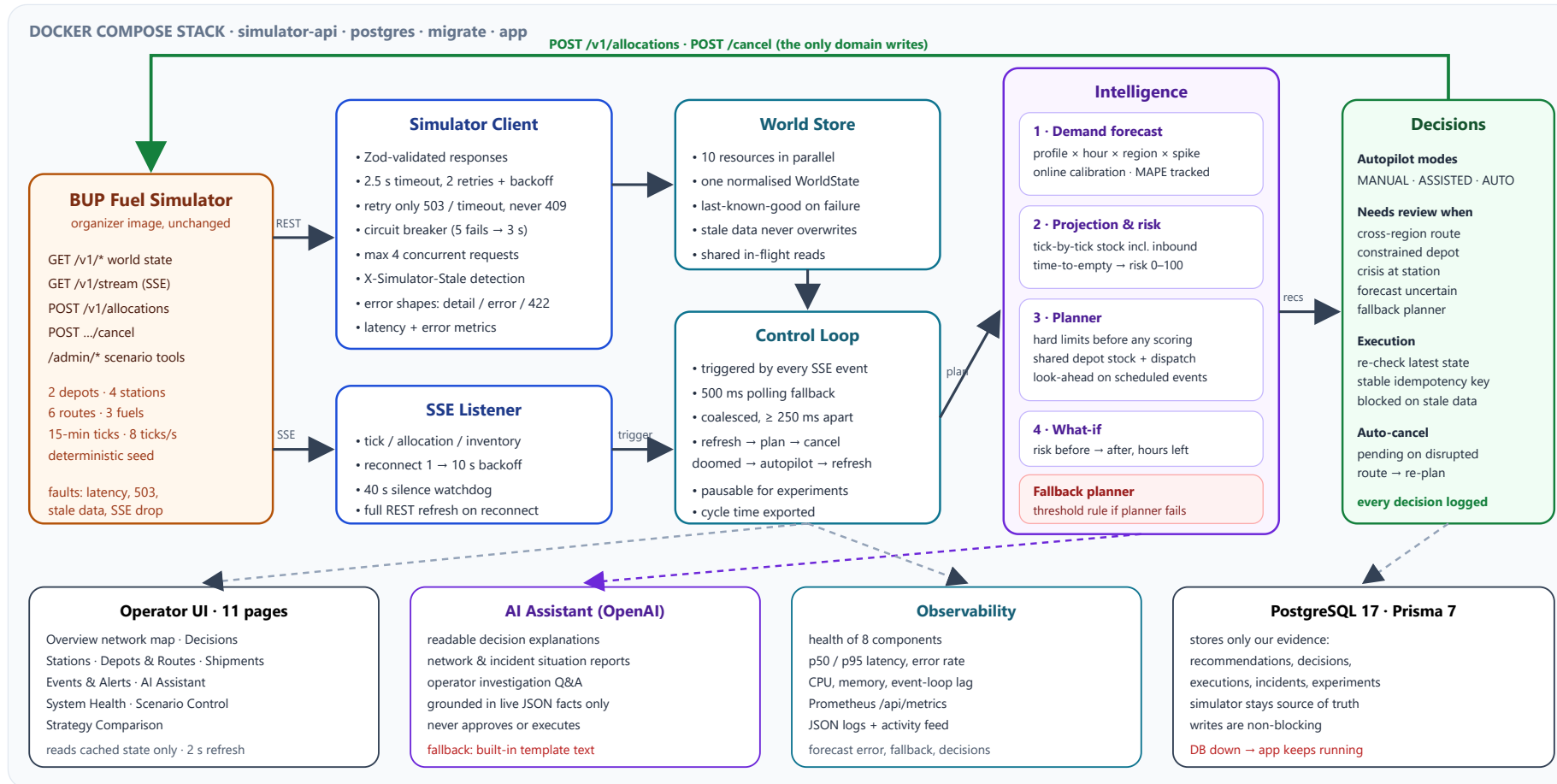
RESULTS (MEASURED, SIMULATOR GROUND TRUTH)

Scenario (192 ticks = 2 days)	No action	Threshold rule	Our planner
Normal operations	46.1%	100%	100%
Combined crisis	42.7%	100%	100%
Severe crisis — service level	33.9%	95.5%	97.7%
Severe crisis — unmet demand	167,734 L	11,377 L	5,877 L (−48%)
Severe crisis — stockout ticks	1,583	38	26

Live run, autopilot AUTO at full speed (8 ticks/s): **100% service level, 0 failed shipments** over 168 ticks and through a combined demand spike + route disruption. Strategies review the network once per simulated hour; same seed and events for every run.

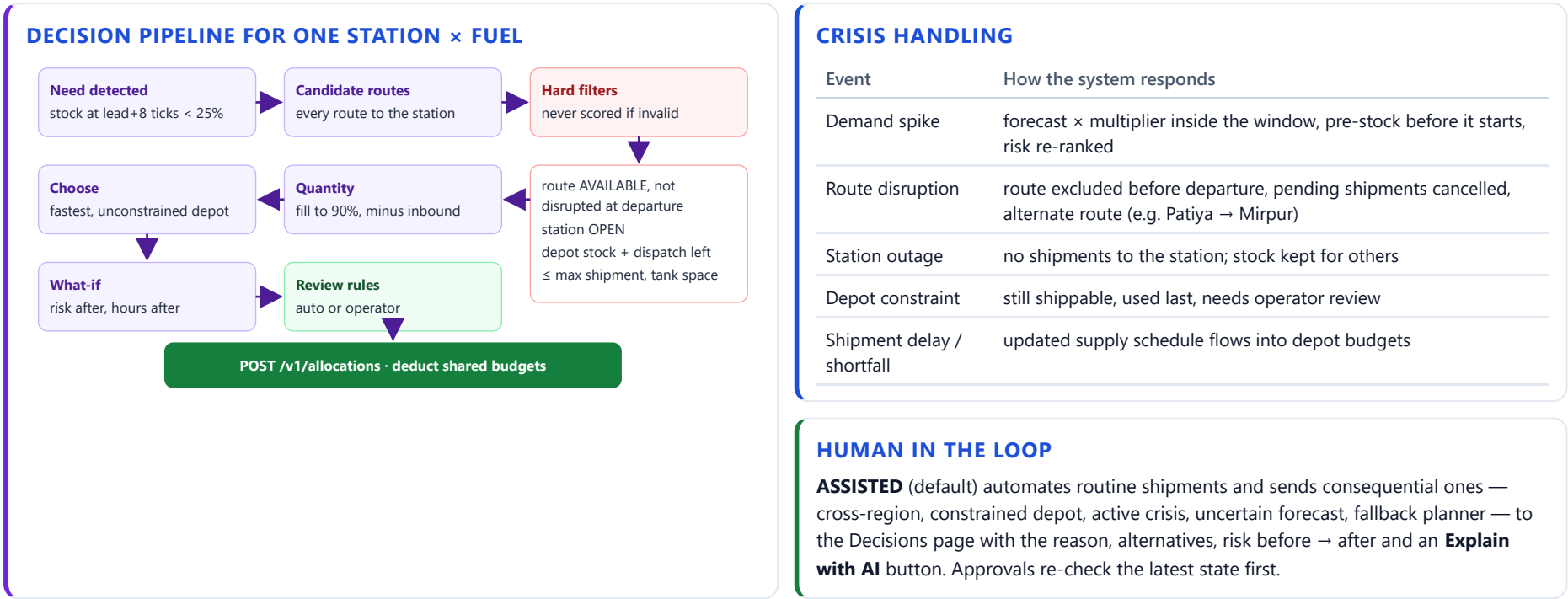
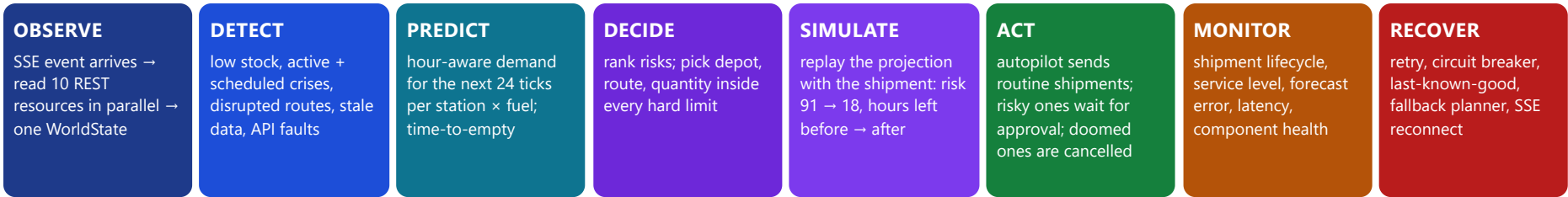
End-to-end architecture

One Next.js application (UI + API + engine) next to the organizer's simulator, orchestrated with Docker Compose. REST is the source of truth; SSE is only a trigger.



What happens every tick — the engineering loop

The brief's loop, implemented as one server-side cycle that runs on every simulator tick (and every 500 ms if the stream is down).



Resilience, deployment and evidence

"Your job is not only to build the system. Your job is to keep it working." Every failure below was injected and observed during development.

FAILURE → DETECTION → BEHAVIOUR

Failure	Detected by	Behaviour
Simulator 503 / error rate	error body <i>error.code</i>	retry with backoff → circuit opens → last-known-good shown with its age
Stale data	X-Simulator-Stale header	STALE banner, good state kept, autopilot paused
Latency	2.5 s timeout	retries, queue ≤ 4 in flight, latency panel amber
SSE disconnect	503 / 40 s silence	reconnect with backoff, 500 ms polling continues
Invalid payload	Zod validation	rejected, alert logged, good state kept
Planner failure	exception	fallback planner, health DEGRADED, operator approval
OpenAI unavailable	timeout / error	built-in template explanations
PostgreSQL down	SELECT 1 probe	health DEGRADED, operations continue in memory
Load on our API	shared cache	UI reads never multiply into simulator load

DEPLOYMENT & DEVOPS

docker compose up -d --build starts four services: `simulator-api` `postgres`

`migrate (one-shot)` `app`

Multi-stage Dockerfile, non-root runtime, health checks, env-only config (no secrets in code). GitHub Actions: lint → typecheck → 32 unit tests → build → compose up → health + API smoke test.

EVIDENCE FOR JUDGES

- **Strategy Comparison** page: re-runs all scenarios on demand and stores results in PostgreSQL.
- **System Health** page + `/api/metrics`: live latency, error rate, resources.
- **k6** script (load/dashboard.js): 100 virtual users on the dashboard path + 20 req/s on the decision API, with p95 < 500 ms thresholds.
- **Scenario Control**: inject any crisis or fault live and watch the platform respond.

ENGINEERING DISCOVERIES WE FIXED

- Health checks proxied to the simulator exhausted its 15-connection pool under load → cached, shared probes + 4-request cap.
- Supply schedule ends at tick 212 → evaluation uses the supplied 2-day window.
- Replayed allocations return 201 (guide says 200) → both accepted.